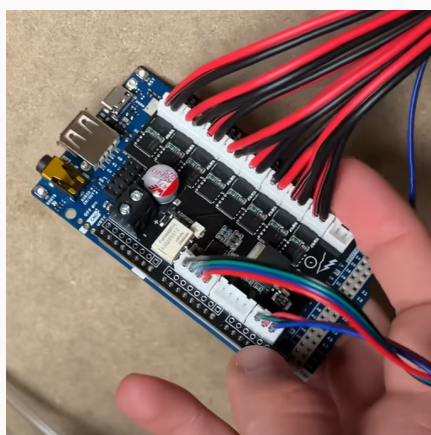
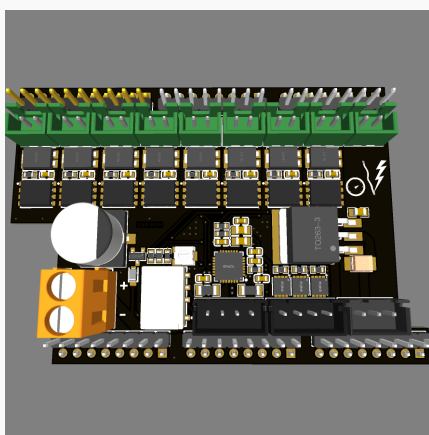
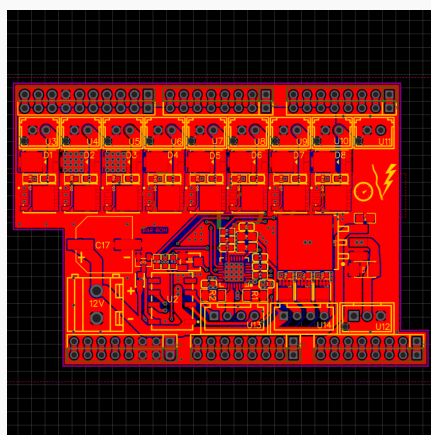


SAMUELE FELICI

Electrical Engineering Student | Università di Catania

 linkedin.com/in/samuelefelici  Telegram: @samuef

ARDUINO® SHIELD FOR DOMOTIC BAR



What?

Designed a custom **Arduino® GIGA shield** to control:

- Eight **DC pumps** with **MOSFETs** and flyback **diodes**
- One **servo motor** using **PWM signal** and **LDO** voltage regulation
- A **stepper motor** using the **TMC2209** driver, plus a **relay** for coil lock when idle
- An **RGB LED strip** controller with **PWM channels** and individual **MOSFET**

How?

Designed the full **schematic** and **PCB layout** in **EasyEDA**, based on **datasheet** reference designs (especially for the **TMC2209**).

Prepared files for factory assembly:

- **Gerber files**
- **Pick-and-place**
- **Bill of Materials (BOM)**

Focused on compact, efficient design tailored to the Arduino® GIGA footprint.

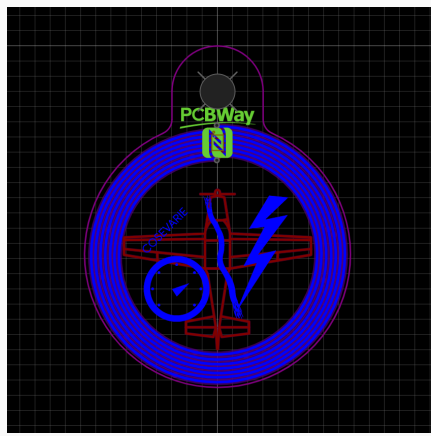
Results

Achieved a fully functional and **space-efficient shield**:

- **Silent stepper operation** with reliable **relay coil lock**
- Maintained **Arduino® GIGA footprint** for easy integration

A detailed third-party **YouTube demo** features this shield, which I designed and delivered for the project.

NFC PCB KEYCHAIN



What?

Designed a custom **NFC-enabled PCB keychain** using the **ST25TN01K** chip from STMicroelectronics.

- Integrated antenna designed for correct **inductance matching**
- **Logo art** on both sides using exposed **ENIG gold-plated traces**
- One side features the creator's logo, the other his airplane symbol

How?

Selected the **ST25TN01K** due to its **fully integrated design**. Antenna was designed from scratch using **inductance calculators** and tuned based on the datasheet specs.

- Created schematic and PCB layout from zero
- Prepared full manufacturing package: **Gerbers, BOM, Pick-and-place**

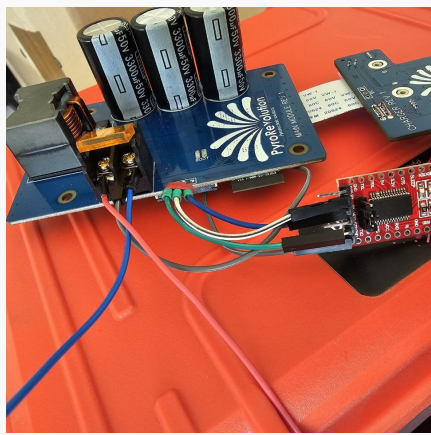
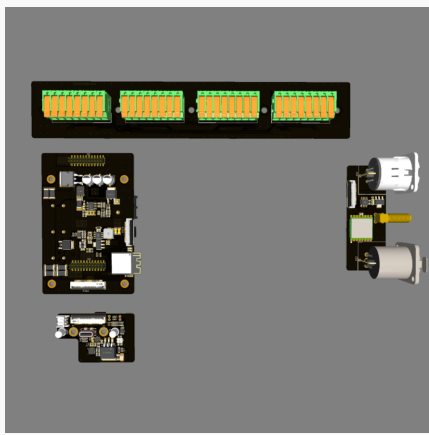
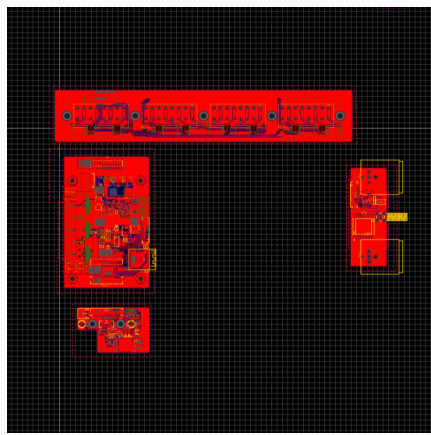
Results

Produced **500+ keychains** with consistent **NFC performance** over long-term use.

- Achieved precise **antenna impedance matching**
- No external passives required, saving cost in bulk production

Demonstrated in this third-party **YouTube video**, featuring the final product

PYROREVOLUTION FIRING MODULE



What?

Designed the **firing module** for the **PyroRevolution** system:

- **32 channels** at **30V DC**
- Dual control: **RS485 (wired)** and **LoRa (wireless)**
- **Battery-powered**, with fast-connect terminals
- **Modular PCB system** to allow easy repairs and upgrades

How?

Fourth revision, fully redesigned from scratch:

- Custom **schematics, PCB layout**, and complete **manufacturing files**
- Selected high-efficiency components (notably a **TI DC-DC driver**)
- Integrated **micro-controller** and **LoRa module**
- Modular design with **pre-soldered threaded standoffs**
- **Firmware** developed in Arduino for real-time operations

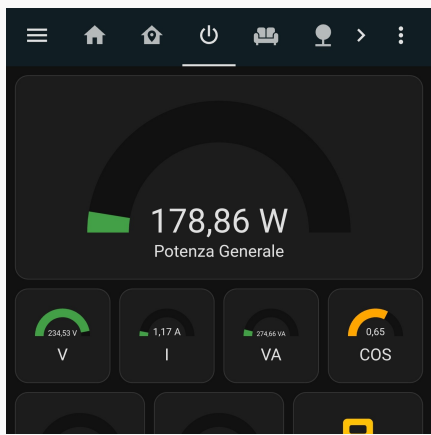
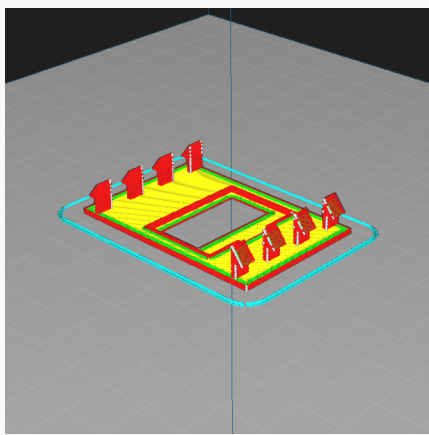
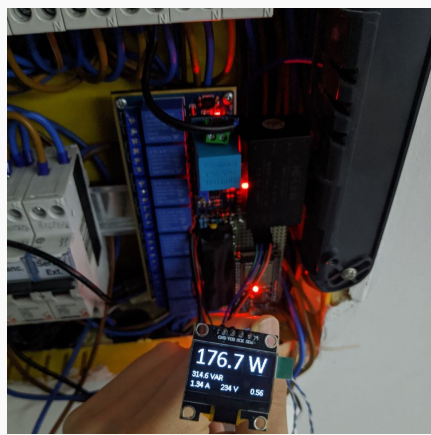
Results

Achieved a **compact, efficient design**:

- **50% smaller** than the previous version
- **4x faster** to assemble, reducing manual soldering
- **Stable and efficient DC-DC conversion**

Designed for scalability and maintenance by non-specialist users

HOME ENERGY METER AND RELAY MODULE



What?

Designed a **home energy meter and relay module** using:

- **ESP32** as the core controller
- **5 current transformers (CT)** for AC current sensing
- **1 voltage transformer** for voltage monitoring
- **8-channel relay board** for load control
- **I2C OLED display** for quick data overview

The device integrates fully with **Home Assistant**.

How?

Firmware developed in **ESPHome**, leveraging the **EmonLib** library for accurate power measurements.

- All components mounted on a **perf-board** (prototype stage)
- **3D-printed mount** for OLED display integration in the power cabinet
- **Laser-cut support** for neat installation

Results

Built a compact and reliable smart energy system:

- Seamless integration with **Home Assistant**
- Real-time current and voltage monitoring
- Full manual and remote control of 8 loads
- Prototyped and deployed in a personal residential setup